

Course: Introduction to Data Analytics with Python.

Instructor: Dr. Andrés García Medina

Email: andgarm.n@gmail.com

Site: <https://sites.google.com/view/andresgmen/home>

Module 1: Introduction to Python and Tools for Data Analytics

Content

- 1: Python installation and configuration: Use of Jupyter/Google Colab and basic built-in methods for data manipulation.
- 2: Business-related example: Import, explore, and export a dataset in the context of business administration.
 - Basics of Pandas
 - Basics of Numpy
 - Basics of Matplotlib
- 3: Upload and open a file from Google Drive

Software

- Python through Google Colab: <https://colab.research.google.com/>

1: Basic Python Syntax

Variables y tipos de datos

```
In [129... # Assign a value to a variable
name = "Juan"
age = 25
height = 1.75

# Print the variables
print("Name:", name, "Age:", age, "Height:", height)

# Another way to print
print(f"Name: {name}, Age: {age}, Height: {height} meters")
```

Name: Juan Age: 25 Height: 1.75
Name: Juan, Age: 25, Height: 1.75 meters

```
In [130...] name, age, height  
name
```

Out[130...] 'Juan'

Basic operations

```
In [131...] # Mathematical Operations  
a = 10  
b = 3  
  
sum = a + b  
subtraction = a - b  
multiplication = a * b  
division = a / b  
  
# Print the results  
print(f"Addition: {sum}, Subtraction: {subtraction}, Multiplication: {multiplication}, Division: {division}")
```

Addition: 13, Subtraction: 7, Multiplication: 30, Division: 3.3333333333333335

```
In [132...] print(sum, subtraction, multiplication, division)
```

13 7 30 3.3333333333333335

You can use python as a calculator

```
In [133...] 10-3
```

Out[133...] 7

```
In [134...] 23*45
```

Out[134...] 1035

```
In [135...] 30/3
```

Out[135...] 10.0

```
In [ ]: 5<=2
```

Out[]: False

```
In [ ]: 3-2
```

Out[]: 1

Print text

```
In [ ]: # This is a comment, Python ignores everything after the symbol #  
print('Hello, world!')
```

Hello, world!

We execute text by adding quotes

```
In [ ]: statement = "we are learning Python for Business"  
print(statement)
```

we are learning Python for Business

The simplest data structure in Python is lists

```
In [ ]: # Defining a list in Python  
my_list = [1, 2, 3, 4, 5]  
print(my_list)
```

[1, 2, 3, 4, 5]

```
In [ ]: # A list can contain different types of data another  
another_list = [1, "two", 3.0, True]  
print(another_list)
```

[1, 'two', 3.0, True]

```
In [ ]: # Access list elements by index (starts at 0)  
print(my_list[0]) # Prints 1  
print(another_list[1]) # Prints "two"
```

1

two

```
In [ ]: # Modify list elements  
my_list[2] = 10  
print(my_list) # Prints [1, 2, 10, 4, 5]
```

[1, 2, 10, 4, 5]

```
In [ ]: # Add items to the list  
my_list.append(6)  
print(my_list) # Prints [1, 2, 10, 4, 5, 6]
```

[1, 2, 10, 4, 5, 6]

```
In [ ]: # Remove items from the list  
my_list.remove(4)  
print(my_list) # Prints [1, 2, 10, 5, 6]
```

[1, 2, 10, 5, 6]

```
In [ ]: # Get the length of the list  
len(my_list) # Prints 5
```

Out[]: 5

In Python, you can use the **insert** method to add an element at a specific location in a list.

The syntax is quite simple: **list.insert(index, value)**.

- **index** is the location where you want the new element to be inserted.
- **value** is the element you want to add.

For example, if you have a list of numbers and you want to add a number in the second place, you would do it like this:

```
In [ ]: numbers = [10, 20, 30, 40]
        numbers.insert(1, 15) # This will insert 15 at index 1
        print(numbers)
```

```
[10, 15, 20, 30, 40]
```

The opposite of insert is pop. While insert adds an element at a specific position in the list, pop removes an element from a list and returns it.

The syntax is list.pop(index).

Index is the location where you want to remove the element. If no index is specified, pop removes and returns the last element in the list.

For example, if you have a list of numbers and you want to remove the number in the third position, you would do the following:

```
In [ ]: numbers = [10, 20, 30, 40]
        removed = numbers.pop(2) # This will remove the element at index 2 (30)
        print(numbers)
        print("Element removed:", removed)
```

```
[10, 20, 40]
```

```
Element removed: 30
```

In Python, you can select parts of many types of sequences (such as lists, tuples, or strings) using what's called slice notation. It's like saying "I want a part of this."

The most basic form of slice notation is **start:stop**, which you pass to the **[]** indexing operator.

start is the index where the slice starts (i.e., the part you want to select). **stop** is the index where the slice ends (but this index is not included).

For example, if you have a list of numbers and you only want the ones between the second and fourth places, you can use slice notation to get those specific values.

```
In [ ]: list = [10, 20, 30, 40, 50]
        cut = list[1:4]
```

```
print(cut)
```

```
[20, 30, 40]
```

In Python, slices are used not only to **read** parts of a list, but also to **assign** new values to those parts. This allows you to modify a section of the list in one go.

```
In [ ]: numbers = [10, 20, 30, 40]
numbers[1:3] = [25, 35] # Swap the elements at positions 1 and 2
print(numbers)
```

```
[10, 25, 35, 40]
```

When working with slices in Python, it's important to remember that the **start index** is included, but the **stop index** is **not** included.

This means that the number of elements you get from a slice is equal to **stop - start**. For example, if you slice **[1:4]**, you get the elements at positions 1, 2, and 3 (not 4).

If you don't specify either the **start** or the **stop**, Python will use the default values: the **start** will be the beginning of the sequence, and the **stop** will be the end of the sequence.

Look at these examples:

```
In [ ]: list = [10, 20, 30, 40, 50]

# Slice from index 1 to 4 (not including 4)
slice = list[1:4] # Gets [20, 30, 40]
print(slice)

# Skip start: start from the beginning
slice_start = list[:3] # Gets [10, 20, 30]
print(slice_start)

# Skip stop: go to the end of the list
slice_stop = list[2:] # Gets [30, 40, 50]
print(slice_stop)

# Skip both start and stop: get the entire list
slice_full = list[:] # Gets [10, 20, 30, 40, 50]
print(slice_full)
```

```
[20, 30, 40]
```

```
[10, 20, 30]
```

```
[30, 40, 50]
```

```
[10, 20, 30, 40, 50]
```

In Python, you can use **negative indices** to slice a sequence relative to the **end** of the list (or tuple, string, etc.).

This is very useful when you want to access elements from back to front without having to count how many elements the sequence has.

- **-1** refers to the **last** element.

- **-2** refers to the **second-to-last** element.
- And so on...

For example:

```
In [ ]: list = [10, 20, 30, 40, 50]

# We use negative indices to get the last elements
last = list[-1] # Gets 50
penultimate = list[-2] # Gets 40
print(last, penultimate)
```

50 40

In Python, when using slicing notation, you can also add a **step** after the colon to take certain elements from the sequence in a more controlled manner.

The **step** allows you to skip some elements in the sequence. For example, you can take every second element, every third element, etc.

The syntax would be:

```
list[start:stop:step]
```

```
In [ ]: list = [10, 20, 30, 40, 50, 60, 70]

# Take all elements, skipping one
slice_step = list[::4]
print(slice_step)
```

[10, 50]

A very useful trick in Python is to use **step** with the value **-1**. This causes the sequence to be traversed in **reversed** order, that is, from back to front.

If you pass **-1** to the slice notation, you can easily reverse a list or tuple, and you don't have to do anything complicated!

Suppose you have a list of numbers and you want to reverse the order of the elements. This is where **-1** comes in:

```
In [ ]: list = [10, 20, 30, 60, 50]

# Reverse the list
reverse_list = list[::-1]
print(reverse_list)
```

[50, 60, 30, 20, 10]

To sort a list

```
In [ ]: my_list = [3, 1, 4, 1, 5, 9, 2, 6]
```

```
# Sort the list in ascending order
my_list.sort()
print(f"List sorted ascending: {my_list}")

# Sort the list in descending order
my_list.sort(reverse=True)
print(f"List sorted descending: {my_list}")
```

List sorted ascending: [1, 1, 2, 3, 4, 5, 6, 9]
 List sorted descending: [9, 6, 5, 4, 3, 2, 1, 1]

The **sort** function has some options that can be very useful in certain cases. One of them is the ability to pass a **secondary sort key**. This means you can use a function that tells you how to sort the objects, based on a value you choose.

For example, suppose you have a list of text strings and you want to sort them not by their content, but by the **length** of the strings. With **sort**, you can do just that: sort them by how many characters they have, instead of sorting by the content of the words.

```
In [ ]: strings = ["apple", "banana", "kiwi", "orange", "grape"]

# Sort strings by length using the len function as the key
strings.sort(key=len)

strings
```

Out[]: ['kiwi', 'apple', 'grape', 'banana', 'orange']

Further built-in data structures in Python are the following:

```
In [ ]: # Tuple: Ordered, immutable, allows duplicate members.
my_tuple = (1, 2, 3, "four")
print("Tuple:", my_tuple)

# Set: Unordered, mutable, no duplicate members.
my_set = {1, 2, 3, 2, 4}
print("Set:", my_set)

# Dictionary: Unordered, mutable, key-value pairs, keys must be unique.
my_dict = {"name": "Alice", "age": 30, "city": "New York"}
print("Dictionary:", my_dict)
```

Tuple: (1, 2, 3, 'four')
 Set: {1, 2, 3, 4}
 Dictionary: {'name': 'Alice', 'age': 30, 'city': 'New York'}

Conditionals and Loops in Python (Advanced)

Conditionals allow you to execute different parts of your code depending on whether a condition is met or not. In Python, this is done with the **if**, **elif** (else if), and **else** keywords.

Imagine you have a number and you want to know if it's positive, negative, or zero. You can use a conditional to check this:

```
In [ ]: number = 5

if number > 0:
    print("The number is positive.")
elif number < 0:
    print("The number is negative.")
else:
    print("The number is zero.")
```

The number is positive.

Another example

```
In [ ]: # Conditional Example
age = 18

if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

You are an adult.

Loops allow you to repeat a piece of code multiple times without having to rewrite it.

A for loop is used when you know how many times you want to repeat something or when you want to iterate through a sequence (such as a list or string). For example, if you have a list of numbers and you want to add them together, you can use a for loop:

```
In [ ]: numbers = [1, 2, 3, 4, 5]
sum = 0

for num in numbers:
    sum += num

print("The sum is:", sum)
```

The sum is: 15

Another example

```
In [ ]: fruits = ["apple", "banana", "cherry", "grape"]

# For loop to iterate over a list
for fruit in fruits:
    print(fruit)
```

apple
banana
cherry
grape

The `while` loop runs as long as a condition is met. It's useful when you don't know how many times you need to repeat an action, but you know it should continue as long as something is true.

```
In [ ]: counter = 0

while counter < 5:
    print("Counter:", counter)
    counter += 1
```

```
Counter: 0
Counter: 1
Counter: 2
Counter: 3
Counter: 4
```

Recap of section 1

- **Basic Python Syntax:**
 - Variables and Data Types (assigning values, printing variables).
 - Basic Mathematical Operations (addition, subtraction, multiplication, division).
 - Printing Text.
- **Data Structures:**
 - **Lists:**
 - Defining lists.
 - Accessing elements by index.
 - Modifying elements.
 - Adding elements (`append` , `insert`).
 - Removing elements (`remove` , `pop`).
 - Getting the length (`len`).
 - Slicing (using `start:stop:step` , including negative indices and reversing).
 - Sorting (`sort` with optional `reverse` and `key` arguments).
 - **Other Built-in Structures:**
 - Tuples (ordered, immutable).
 - Sets (unordered, mutable, no duplicates).
 - Dictionaries (unordered, mutable, key-value pairs).
- **Conditionals:**
 - Using `if` , `elif` , and `else` to control code flow based on conditions.
- **Loops:**
 - **for loops:** Iterating over sequences (like lists) or for a known number of repetitions.
 - **while loops:** Repeating code as long as a condition is True.

Specific Commands/Methods:

- `print()` : Outputting text and variable values.
- `len()` : Getting the length of sequences (like lists).
- `.append()` : Adding an element to the end of a list.
- `.insert(index, value)` : Inserting an element at a specific index in a list.
- `.remove(value)` : Removing the first occurrence of a specific value from a list.
- `.pop(index)` : Removing and returning an element at a specific index (or the last element if no index is given) from a list.
- `.sort()` : Sorting a list in place.

2: Guide Example: Restaurant Tip Data

Characteristics of the `tips` dataset

Column	Description
<code>total_bill</code>	Total bill in dollars.
<code>tip</code>	Tip in dollars.
<code>sex</code>	Sex of the bill holder (<code>Male</code> or <code>Female</code>).
<code>smoker</code>	Indicates whether there are smokers in the group (<code>Yes</code> or <code>No</code>).
<code>day</code>	Day of the week (<code>Thur</code> , <code>Fri</code> , <code>Sat</code> , <code>Sun</code>).
<code>time</code>	Time of day (<code>Lunch</code> or <code>Dinner</code>).
<code>size</code>	Size of the group of diners.

The data were reported in a collection of case studies for business statistics (Bryant & Smith 1995).

2.1. Explore Data with Pandas

This module contains data structures and data manipulation tools designed to make data cleaning and analysis quick and easy in Python.

The first step is to import the `Pandas` module for data analysis and manipulation.

```
In [ ]: import pandas as pd
```

We use the pandas function `read_csv` to read the data from a CSV file located in an open access repository and assign it to a variable that we name (arbitrarily) `datos_tips`:

```
In [ ]: data_tips = pd.read_csv("https://raw.githubusercontent.com/mwaskom/\nseaborn-data/master/tips.csv")
```

We show the first rows of the *dataset*

```
In [ ]: data_tips.head()
```

```
Out[ ]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

We request general information about the *dataset*

```
In [ ]: data_tips.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   total_bill  244 non-null   float64
1   tip         244 non-null   float64
2   sex        244 non-null   object
3   smoker     244 non-null   object
4   day        244 non-null   object
5   time       244 non-null   object
6   size       244 non-null   int64
dtypes: float64(2), int64(1), object(4)
memory usage: 13.5+ KB
```

Calculamos las principales estadísticas descriptivas con el método **describe**

```
In [ ]: data_tips.describe()
```

```
Out[ ]:
```

	total_bill	tip	size
count	244.000000	244.000000	244.000000
mean	19.785943	2.998279	2.569672
std	8.902412	1.383638	0.951100
min	3.070000	1.000000	1.000000
25%	13.347500	2.000000	2.000000
50%	17.795000	2.900000	2.000000
75%	24.127500	3.562500	3.000000
max	50.810000	10.000000	6.000000

We can save the results to a file with a CSV extension using the **to_csv** method:

```
In [ ]: data_tips.to_csv('table_tips.csv', index=False)
# index=False to avoid saving the index
# Annotations that begin with the 'hashtag' symbol are not executed
```

The file **table_tips.csv** has been saved in Google Colab's temporary storage, so we should download it manually from the left panel.

A more sophisticated option is to import the **files** function from the **google.colab** module, and give the following instruction to download the file automatically:

```
In [ ]: from google.colab import files
files.download('table_tips.csv')
```

If we prefer to save the table in Excel's XLSX format, we can write the following:

```
In [ ]: data_tips.to_excel('table_tips.xlsx', index=False)
```

and again to download the file automatically

```
In [ ]: files.download('table_tips.xlsx')
```

Let's now suppose we want to upload a file from our computer.

For the purposes of this example, we'll upload the same XLSX file we just downloaded, but the idea applies to any file with this extension.

Simply type the following and search for the desired file in your documents:

```
In [ ]: uploaded = files.upload()
```

Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving table_tips.xlsx to table_tips (1).xlsx

To verify that it has been loaded correctly, we print the first lines

```
In [ ]: tips = pd.read_excel("table_tips.xlsx")
tips.head()
```

```
Out[ ]:
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

De igual forma con `read_csv`

```
In [ ]: uploaded = files.upload()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving table_tips.csv to table_tips (2).csv

```
In [ ]: propinas = pd.read_csv("table_tips.csv")
propinas.head()
```

```
Out [ ]:   total_bill  tip  sex  smoker  day  time  size
0      16.99  1.01  Female     No  Sun  Dinner    2
1      10.34  1.66   Male     No  Sun  Dinner    3
2      21.01  3.50   Male     No  Sun  Dinner    3
3      23.68  3.31   Male     No  Sun  Dinner    2
4      24.59  3.61  Female     No  Sun  Dinner    4
```

It is also possible to use `read_table` to open the data if it is in CSV format

```
In [ ]: tips = pd.read_table("table_tips.csv", sep=',')
tips.head()
```

```
Out [ ]:   total_bill  tip  sex  smoker  day  time  size
0      16.99  1.01  Female     No  Sun  Dinner    2
1      10.34  1.66   Male     No  Sun  Dinner    3
2      21.01  3.50   Male     No  Sun  Dinner    3
3      23.68  3.31   Male     No  Sun  Dinner    2
4      24.59  3.61  Female     No  Sun  Dinner    4
```

But, with `read_table` we need to specify the `sep` type explicitly

If necessary, we can omit the header and consider it as just another record.

```
In [ ]: tips = pd.read_csv("table_tips.csv", header=None)
tips.head()
```

```
Out[ ]:
```

	0	1	2	3	4	5	6
0	total_bill	tip	sex	smoker	day	time	size
1	16.99	1.01	Female	No	Sun	Dinner	2
2	10.34	1.66	Male	No	Sun	Dinner	3
3	21.01	3.5	Male	No	Sun	Dinner	3
4	23.68	3.31	Male	No	Sun	Dinner	2

2.2. Explore data with NumPy (advanced)

NumPy, short for Numerical Python, is one of the most important fundamental packages for numerical computation in Python.

Aunque el módulo **NumPy** está más enfocado al análisis numérico, también podemos cargar, manipular, y guardar datos de manera equivalente a como hicimos con el módulo **Pandas**.

El primer paso es importar **NumPy**

```
In [ ]: import numpy as np
```

Define column names (according to the CSV file)

```
In [ ]: columns_names = ['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size']
```

We load the data from the URL (similar to `pd.read_csv`)

```
In [ ]: data_url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
```

```
In [ ]: data_tips = np.genfromtxt(data_url, delimiter=',', skip_header=1, dtype=None, names=columns_names, encoding='utf-8')
```

We display the first rows of the *dataset* in a similar way to `datos_tips.head()`

```
In [ ]: print(data_tips[:5])
```

```
[(16.99, 1.01, 'Female', 'No', 'Sun', 'Dinner', 2)
 (10.34, 1.66, 'Male', 'No', 'Sun', 'Dinner', 3)
 (21.01, 3.5, 'Male', 'No', 'Sun', 'Dinner', 3)
 (23.68, 3.31, 'Male', 'No', 'Sun', 'Dinner', 2)
 (24.59, 3.61, 'Female', 'No', 'Sun', 'Dinner', 4)]
```

To obtain general information about the *dataset*, we can request the data type of each column. However, this method is a bit more limited compared to **Pandas**

```
In [ ]: for column in data_tips.dtype.names:
         print(f"Column: {column}, Type: {data_tips.dtype[column]}")
```

```
Column: total_bill, Type: float64
Column: tip, Type: float64
Column: sex, Type: <U8
Column: smoker, Type: <U5
Column: day, Type: <U6
Column: time, Type: <U8
Column: size, Type: int64
```

Similarly, it is possible to calculate each of the descriptive statistics for the dataset.

In **NumPy**, we need to manually select the numeric columns: 0 (total_bill), 1 (tip), and 6 (size), and transform them into a 2D matrix.

```
In [ ]: data_tips_numericos = np.array([data_tips['total_bill'], data_tips['tip'],\
                                         data_tips['size']]).T
```

We see the first 10 rows of the resulting (numeric) data matrix:

```
In [ ]: print(data_tips_numericos[:10,:])
```

```
[[16.99  1.01  2.  ]
 [10.34  1.66  3.  ]
 [21.01  3.5   3.  ]
 [23.68  3.31  2.  ]
 [24.59  3.61  4.  ]
 [25.29  4.71  4.  ]
 [ 8.77  2.    2.  ]
 [26.88  3.12  4.  ]
 [15.04  1.96  2.  ]
 [14.78  3.23  2.  ]]
```

It is now possible to apply each of the descriptive statistics.

To count non-zero values, we use **count_nonzero**

```
In [ ]: np.count_nonzero(data_tips_numericos, axis=0)
# axis=0 specifies that we count across all rows
# use axis=1 if you want to count across all columns
```

```
Out[ ]: array([244, 244, 244])
```

To calculate the average over all row values we use

```
In [ ]: np.mean(data_tips_numericos, axis=0)
```

```
Out[ ]: array([19.78594262,  2.99827869,  2.56967213])
```

For the standard deviation

```
In [ ]: np.std(data_tips_numericos, axis=0)
```

```
Out[ ]: array([8.88415058, 1.38079995, 0.94914883])
```

If we want to know the minimum value

```
In [ ]: np.min(data_tips_numericos, axis=0)
```

```
Out[ ]: array([3.07, 1.  , 1.  ])
```

The maximum value

```
In [ ]: np.max(data_tips_numericos, axis=0)
```

```
Out[ ]: array([50.81, 10.  , 6.  ])
```

and to obtain the mean, first and third quartile as follows:

```
In [ ]: np.median(data_tips_numericos, axis=0),
```

```
Out[ ]: (array([17.795, 2.9  , 2.  ]),)
```

```
In [ ]: np.percentile(data_tips_numericos, 25, axis=0),
```

```
Out[ ]: (array([13.3475, 2.  , 2.  ]),)
```

```
In [ ]: np.percentile(data_tips_numericos, 75, axis=0),
```

```
Out[ ]: (array([24.1275, 3.5625, 3.  ]),)
```

On the other hand, with **NumPy** you can save in text or binary format, but not in CSV directly.

Suppose we are interested in saving the results of the average of each column, the first step is to save the output in a variable that we can name arbitrarily as **averages**

```
In [ ]: averages = np.mean(data_tips_numericos, axis=0)
print(averages)
```

```
[19.78594262  2.99827869  2.56967213]
```

Now we save the results in a CSV file with the **savetxt** function

```
In [ ]: np.savetxt('average_results.csv', averages, delimiter=',', fmt='%s')
```

and we automatically download the file to our files

```
In [ ]: files.download('average_results.csv')
```

If we want to load a data file from our computer

```
In [ ]: uploaded = files.upload()
```


Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving `average_results.csv` to `average_results (1).csv`

To verify that it has been loaded correctly, we print its values.

```
In [ ]: data = np.genfromtxt('average_results.csv', delimiter=',')
        print(data)
```

```
[19.78594262  2.99827869  2.56967213]
```

2.3. Explore data with `matplotlib`

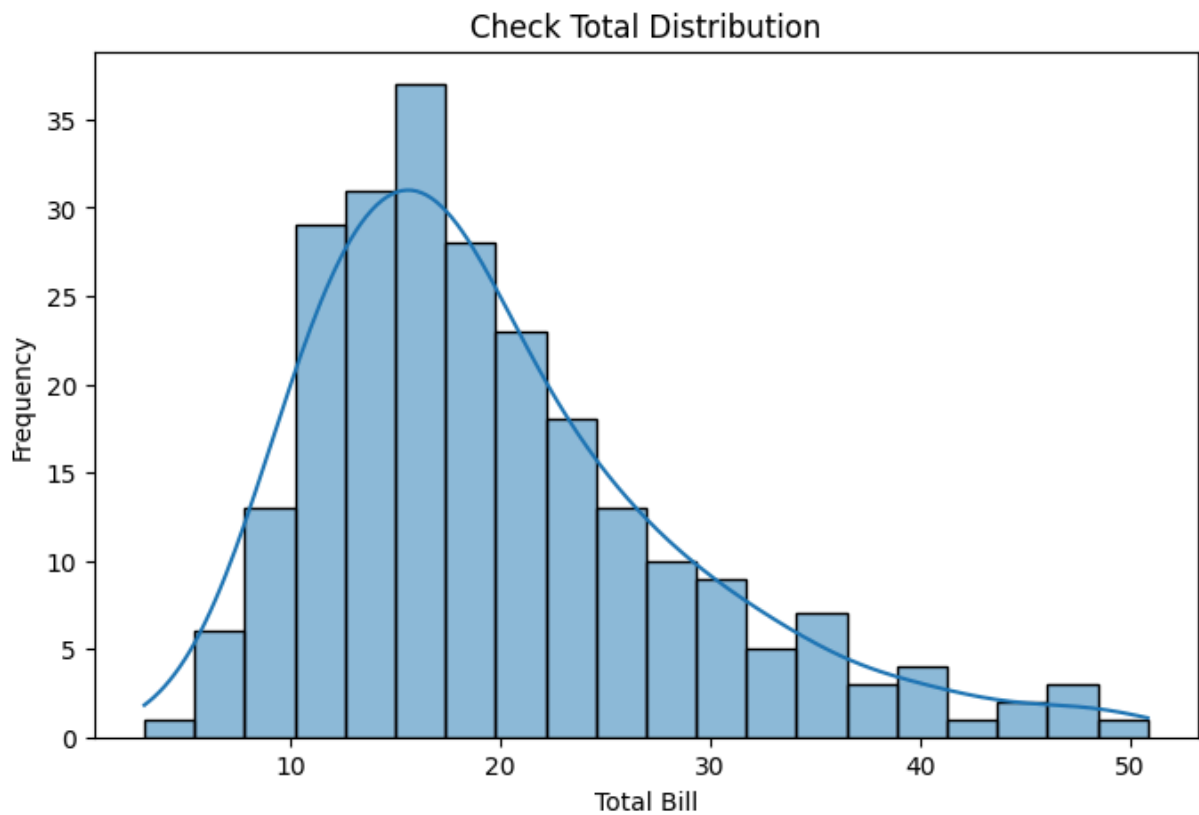
- Creating informative visualizations (sometimes called charts) is one of the most important tasks in data analysis.
- It can be part of the exploratory process, for example, to help identify outliers or necessary data transformations, or it can serve as a tool to generate insights in model building.
- In other cases, creating an interactive visualization for the web may be the ultimate goal.
- Python has many additional libraries for creating static or dynamic visualizations. However, in this course, we will focus primarily on `matplotlib` and the libraries built on top of it, particularly `seaborn`.
- In this section, we will focus on generating and saving charts.

The first step is to import the `matplotlib` and `seaborn` modules for data visualization and understanding.

```
In [ ]: import matplotlib.pyplot as plt
        import seaborn as sns
```

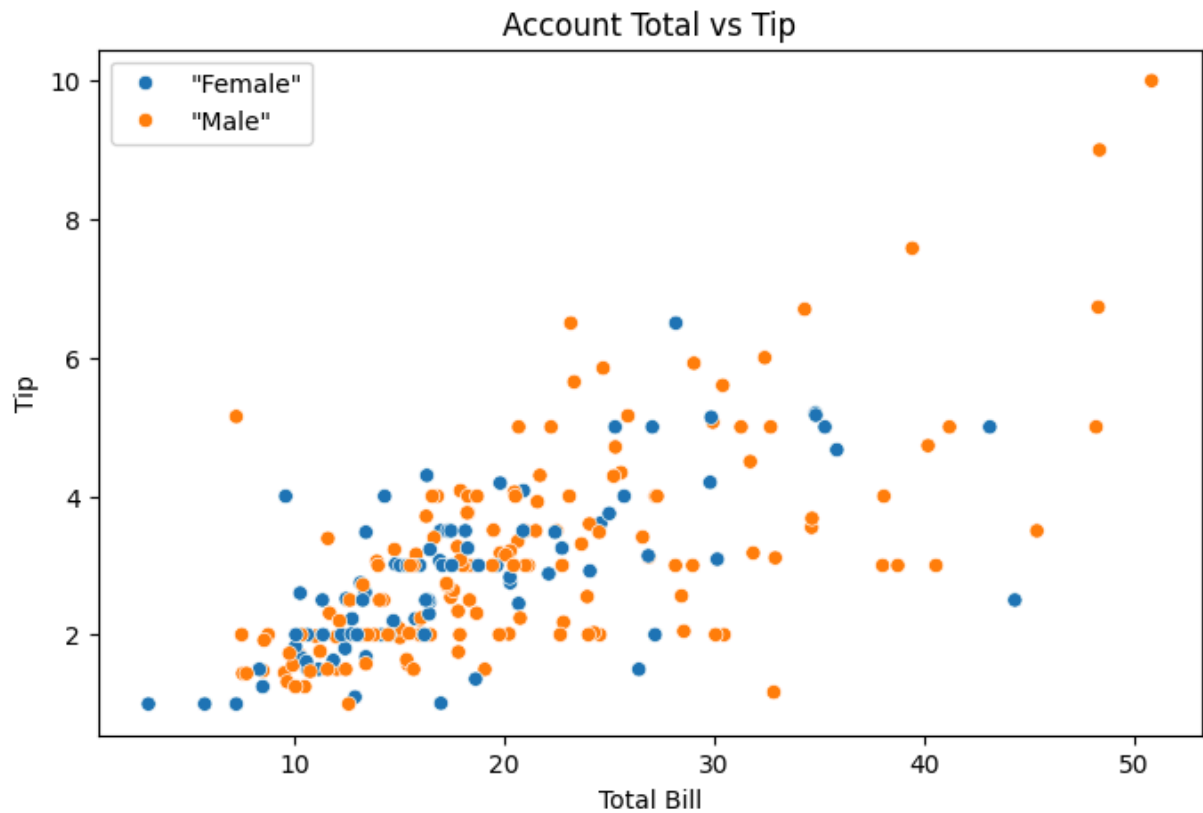
As an example, let's generate the total distribution of the bills (*total_bill*) from the file that contains the tip data.

```
In [ ]: # Histogram of check totals
        plt.figure(figsize=(8, 5)) #inches
        sns.histplot(data_tips['total_bill'], bins=20, kde=True)
        plt.title("Check Total Distribution")
        plt.xlabel("Total Bill")
        plt.ylabel("Frequency")
        plt.show()
```



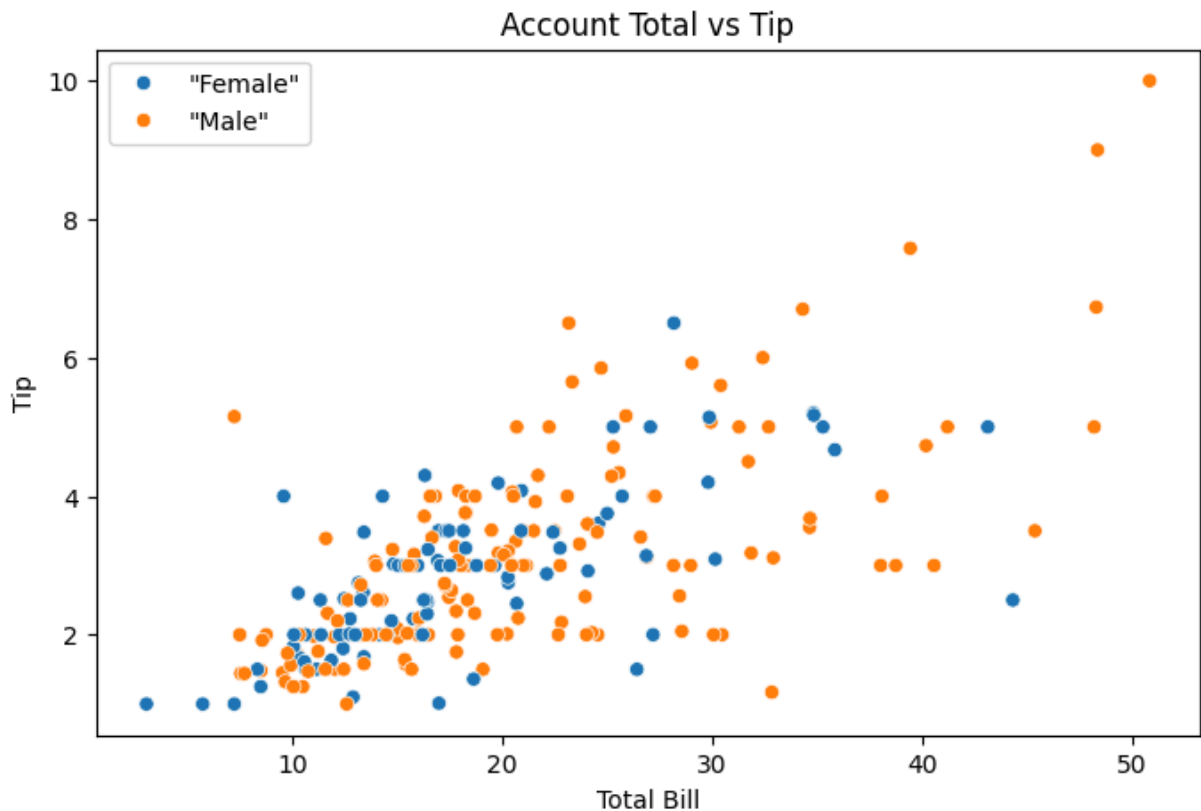
We can analyze the relationship between total bill and tips

```
In [ ]: plt.figure(figsize=(8, 5))
sns.scatterplot(x=data_tips['total_bill'], y=data_tips['tip'],\
                hue=data_tips['sex'])
plt.title("Account Total vs Tip")
plt.xlabel("Total Bill")
plt.ylabel("Tip")
plt.show()
```



To save the graph we need to add the instruction `savefig` before displaying the figure

```
In [ ]: plt.figure(figsize=(8, 5))
sns.scatterplot(x=data_tips['total_bill'], y=data_tips['tip'],\
                hue=data_tips['sex'])
plt.title("Account Total vs Tip")
plt.xlabel("Total Bill")
plt.ylabel("Tip")
plt.savefig("tip_graph.png")
plt.show()
```



Finally, to download the image automatically we use again the `download` function of the `files` method

```
In [ ]: files.download("tip_graph.png")
```

Recap of section 2

Business-related example: Import, explore, and export a dataset in the context of business administration, focuses on applying basic Python data handling techniques to a real-world business scenario using a restaurant tip dataset.

1. **Dataset Introduction:** The section introduces the `tips` dataset, describing its columns: `total_bill`, `tip`, `sex`, `smoker`, `day`, `time`, and `size`.

2. Exploring Data with Pandas:

- **Importing Data:** Demonstrates how to read a CSV file from a URL using `pd.read_csv()`.
- **Data Inspection:** Shows how to view the first few rows (`.head()`), get summary information about columns and data types (`.info()`), and calculate descriptive statistics (`.describe()`).
- **Exporting Data:** Explains how to save a DataFrame to a CSV file (`.to_csv()`) and an Excel file (`.to_excel()`). It also includes how to download these files from Google Colab using `files.download()`.

- **Uploading Data:** Shows how to upload files from your local computer to Google Colab using `files.upload()` and then read them into a Pandas DataFrame using `pd.read_excel()`, `pd.read_csv()`, and `pd.read_table()` (highlighting the need to specify the delimiter with `read_table()`). It also mentions the option to read data without a header using the `header=None` argument in `read_csv`.

3. Exploring Data with NumPy (Advanced):

- **Importing Data:** Shows how to load data from a URL into a NumPy structured array using `np.genfromtxt()`, specifying delimiters, skipping headers, data types, and column names.
- **Data Inspection:** Demonstrates how to print the first few rows of a NumPy array and iterate through its data types.
- **Descriptive Statistics:** Explains how to select numeric columns, create a 2D NumPy array from them, and calculate descriptive statistics like non-zero counts (`np.count_nonzero()`), mean (`np.mean()`), standard deviation (`np.std()`), minimum (`np.min()`), maximum (`np.max()`), median (`np.median()`), and percentiles (`np.percentile()`), often specifying the axis (`axis=0` for columns, `axis=1` for rows).
- **Exporting Data:** Shows how to save NumPy arrays to text files (which can be used for CSV) using `np.savetxt()` and how to download them.
- **Uploading Data:** Demonstrates how to upload files and load them into a NumPy array using `np.genfromtxt()`.

4. Exploring Data with Matplotlib:

- **Importing Libraries:** Mentions importing `matplotlib.pyplot` and `seaborn` for visualization.
- **Creating Visualizations:** Provides examples of creating visualizations:
 - A histogram of `total_bill` using `seaborn.histplot()`.
 - A scatter plot showing the relationship between `total_bill` and `tip`, colored by `sex`, using `seaborn.scatterplot()`.
- **Customizing Plots:** Shows how to set figure size, add titles (`plt.title()`), and label axes (`plt.xlabel()`, `plt.ylabel()`).
- **Displaying Plots:** Uses `plt.show()` to display the generated plots.
- **Saving Plots:** Explains how to save plots to files (e.g., PNG) using `plt.savefig()` before calling `plt.show()`.
- **Downloading Plots:** Shows how to automatically download the saved image files using `files.download()`.

In essence, Module 2 provides a practical application of basic data handling in Python using the Pandas, NumPy, and Matplotlib libraries within the Google Colab environment, simulating a typical workflow of importing, exploring, and exporting a dataset relevant to a business context.

3: Upload and open a file from Google Drive

- The first step is to set up Drive in Google Collab

```
In [114... from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

- Next we determine our working directory

```
'/content/drive/MyDrive/'
```

- En esta carpeta se ubican las subcarpetas personales.
- For example, in my personal Google Drive cloud, I've created these folders and subfolders:

```
/Business_Statistics_2025Q3/Workshop/Notebooks/Introduction_Python_Data_Analytics_Tool
```

- And within the "Modulo_1" subfolder, I have saved this Google Colab workbook.
- So my full working directory is as follows:

```
'/content/drive/Business_Statistics_2025Q3/Workshop/Notebooks/'
```

- Each user must locate or create a personal work folder to save their workbook and then assign it to a Python variable similar to the following:



```
In [119... personal_folder='/content/drive/MyDrive/Business_Statistics_2025Q3/Workshop/
```

- Once this is done, you must change your working directory to the personal folder defined above (it will be different for each one) as follows:

```
In [120... import os
os.chdir(personal_folder)
```

- You can even verify that you are in your home folder with the following command (optional):

```
In [124... os.getcwd()
```

```
Out[124... '/content/drive/MyDrive/Business_Statistics_2025Q3/Workshop/Notebooks'
```

- If you get an error, you should carefully check that you have entered the folder names exactly as they appear in your Drive.
- Python (and any programming language) is very sensitive to spaces, capital letters, accents, and other characters.
- If all goes well, you can now continue loading the data.

Example

- Let's now continue by loading and graphing the weighted average price of gasoline per gallon in the United States, based on a sample of approximately 900 retail outlets, as of 8:00 a.m. Monday, from 1990 to 2024.
- This information can be found publicly at the following link:
<https://fred.stlouisfed.org/series/GASREGW>
- You can practice by manually downloading the information from the website and uploading it to a personal folder in Drive.
- In my case I have loaded it in the same previous work folder

```
In [125... import pandas as pd
datos_gasolina = pd.read_csv('GASREGW.csv')
print(datos_gasolina)
```

	observation_date	GASREGW
0	1990-08-20	1.191
1	1990-08-27	1.245
2	1990-09-03	1.242
3	1990-09-10	1.252
4	1990-09-17	1.266
...	...	...
1816	2025-06-09	3.108
1817	2025-06-16	3.139
1818	2025-06-23	3.213
1819	2025-06-30	3.164
1820	2025-07-07	3.125

[1821 rows x 2 columns]

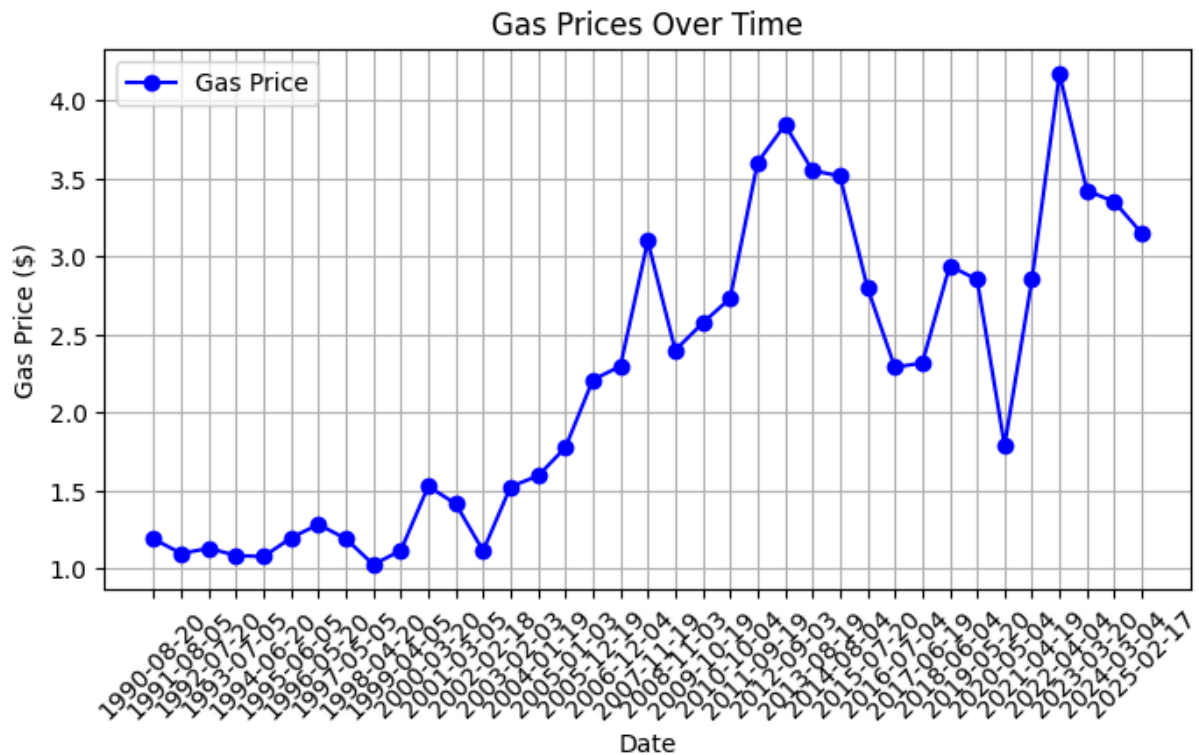
```
In [127... import matplotlib.pyplot as plt

# Plot
plt.figure(figsize=(8, 4))
plt.plot(datos_gasolina["observation_date"][::50], \
         datos_gasolina["GASREGW"][::50], marker='o', \
         linestyle='--', color='b', label="Gas Price")
```

```
# Formatting
plt.xlabel("Date")
plt.ylabel("Gas Price ($)")
plt.title("Gas Prices Over Time")
plt.legend()
plt.grid(True)

# Rotate x-axis labels for readability
plt.xticks(rotation=45)

# Show plot
plt.savefig('gasoline.png', dpi=300)
plt.show()
```



The figure was saved in the Google Drive working folder, which we can corroborate with the following instruction (command):

In [128...

```
ls
```

```
'Análisis Descriptivo y Visualización de Datos.ipynb'
gasoline.png
GASREGW.csv
Introducción_al_Análisis_Predictivo.ipynb
'Introduction to Python and Data Analytics Tools.ipynb'
'Manipulación de Datos con Pandas.ipynb'
```

Recap of section 3

1. **Mounting Google Drive:** The essential first step is to connect Colab to your Google Drive account using `drive.mount('/content/drive')`.

2. **Identifying Working Directory:** It explains that `/content/drive/MyDrive/` is the base directory for your personal Drive content in Colab. Users need to locate or create a specific folder within their Drive where they store their data files and notebooks.
3. **Changing Working Directory:** It demonstrates how to change the current working directory in Colab to the specified personal folder in Google Drive using `os.chdir()`. This makes it easier to access files without typing the full path every time. An optional step `os.getcwd()` is provided to verify the current directory. It highlights the importance of exact spelling and case sensitivity when specifying folder names.
4. **Example:** A practical example is provided where a CSV file (`GASREGW.csv`) containing historical gasoline price data, assumed to be stored in the user's specified Drive folder, is loaded into a Pandas DataFrame using `pd.read_csv()`.
5. **Plotting Data from Drive:** The example then proceeds to create a line plot of the gasoline prices over time using `matplotlib.pyplot`, demonstrating that data loaded from Drive can be used for analysis and visualization.
6. **Saving Plots to Drive:** It shows how to save the generated plot image directly back to the working folder in Google Drive using `plt.savefig()`.
7. **Verifying Files in Drive:** Finally, the shell command `!ls` is used to list the contents of the current directory, confirming that the saved image file is now present in the specified Google Drive folder.

In summary, Section 3 provides a step-by-step guide on how to integrate Google Drive with Google Colab, enabling users to easily access, process, and save files directly within their cloud storage environment.

General Summary

- Section 1 covers fundamental Python concepts and essential tools for data analytics. It introduces Python installation using Jupyter/Google Colab, basic syntax (variables, data types, operations, printing), fundamental data structures like lists (including operations like append, insert, remove, pop, slicing, and sorting), tuples, sets, and dictionaries. It also covers control flow with conditional statements (`if`, `elif`, `else`) and loops (`for`, `while`).
- Section 2 focuses on a practical business example using a restaurant tip dataset. It demonstrates how to import, explore, and export data using the Pandas library, including reading CSV/Excel files, inspecting data with `.head()`, `.info()`, and `.describe()`, and saving data using `.to_csv()`, `.to_excel()`, and `.savetxt()`. It also introduces basic data exploration and manipulation with NumPy (though noted as more advanced) and data visualization using Matplotlib and Seaborn, including creating histograms and scatter plots and saving figures.

- Section 3 guides users on how to integrate Google Drive with Google Colab. It explains how to mount Drive, identify and change the working directory within Drive, and then demonstrates loading data files directly from a specified Drive folder into a Pandas DataFrame. It concludes by showing how to create a plot using this data and save the resulting image file back to the Google Drive folder.

Referencias

- Anderson, D. R., Williams, T. A. y Cochran, J. J. (2024). Estadística para la economía y los negocios. Cengage Learning.
- McKinney, W. (2012). Python para el análisis de datos: Manejo de datos con Pandas, NumPy e IPython. "O'Reilly Media, Inc."
- Bryant, P. G. y Smith, M. (1995) *Análisis práctico de datos: Casos prácticos en estadística empresarial*. Homewood, IL: Richard D. Irwin Publishing.
- OpenAI. (2025). *ChatGPT (versión GPT-4)*. Recuperado de <https://chat.openai.com/>